relationships between spaces, devices, and sensors while delivering data in formats that both technical systems and end users can readily consume.

DATA ACCESS AND QUERY EFFICIENCY: REST, ODATA, AND GRAPHQL

As we have seen, digital twins operate on knowledge graphs—interconnected webs of relationships linking physical spaces, smart devices, sensor networks, and continuous measurement streams. The protocol we choose to expose this data affects both system performance and developer experience. Table 5.1 compares 3 popular API protocols for retrieving data over HTTP that can be adopted to serve data from a digital twin.

Table 5.1 A comparison of API protocols that can be used to serve data from a digital twin.

Protocol	Architecture	Query mechanism	Efficiency for complex data
REST	Multiple resource-specific endpoints, for example `/rooms`, `/sensors`	Fixed server responses; rigid data shape	**Low**. Requires multiple sequential requests
ODATA	Standardized protocol built on REST principles	Flexible querying via URL parameters (`$filter`, `$select`, `$expand`)	**Moderate**. Allows clients to request related data in a single request using `$expand`, improving efficiency over REST
GraphQL	Single endpoint (`$graphql`)	Client-defined query language specifying exact data needs	**High**. Addresses over-fetching; retrieves all required data across relationships in a single round trip

MULTI-STEP DATA FETCHING

Given the complexity of data relationships in a digital twin's model of reality, the way in which your API retrieves data is important to consider, both in terms of performance and usability of your interface.